

TEA Algorithm - Allen Jue

One Step of TEA Encryption

```
(defun tea-encrypt-step (v0 v1 k0 k1 k2 k3 sum)
  (declare (xargs :guard (... check if u32)))
  (let* ((sum (mod (+ sum #x9E3779B9) #x100000000))
        (v0 (mod (+ v0 (logxor (+ (ash v1 4) k0)
                                (+ v1 sum)
                                (+ (ash v1 -5) k1)))
                  #x100000000))
        (v1 (mod (+ v1 (logxor (+ (ash v0 4) k2)
                                (+ v0 sum)
                                (+ (ash v0 -5) k3)))
                  #x100000000)))
    (mv v0 v1 sum)))
```

Does Multiple Rounds of TEA Encryption

```
(defun tea-encrypt (v0 v1 k0 k1 k2 k3 rounds sum)
  (declare (xargs :guard (... check if u32)))
  (if (zp rounds)
      (mv v0 v1 sum)
      (mv-let (new-v0 new-v1 new-sum)
        (tea-encrypt-step v0 v1 k0 k1 k2 k3 sum)
        (tea-encrypt new-v0 new-v1 k0 k1 k2 k3 (- rounds 1)
                      new-sum))))
```

One Step of TEA Decryption

```
(defun tea-decrypt-step (v0 v1 k0 k1 k2 k3 sum)
  (declare (xargs :guard (... check if u32)))
  (let* ((v1 (mod (- v1 (logxor (+ (ash v0 4) k2)
                                (+ v0 sum)
                                (+ (ash v0 -5) k3)))
                  #x100000000))
        (v0 (mod (- v0 (logxor (+ (ash v1 4) k0)
                                (+ v1 sum)
                                (+ (ash v1 -5) k1)))
                  #x100000000))
        (sum (mod (- sum #x9E3779B9) #x100000000)))
    (mv v0 v1 sum)))
```

Does Multiple Rounds of TEA Decryption

```
(defun tea-decrypt (v0 v1 k0 k1 k2 k3 rounds sum)
  (declare (xargs :guard (... check if u32)))
  (if (zp rounds)
      (mv v0 v1 sum)
      (mv-let (new-v0 new-v1 new-sum)
        (tea-decrypt-step v0 v1 k0 k1 k2 k3 sum)
        (tea-decrypt new-v0 new-v1 k0 k1 k2 k3 (- rounds 1)
                      new-sum))))
```

TEA Step Invertibility, Test Cases, Potential Directions

One Step Invertibility

(defthm tea-step-invertible

(implies (All are u32...))

;; save the results of 1 step of encryption

(mv-let (v0-new v1-new sum-new)

(tea-encrypt-step v0 v1 k0 k1 k2 k3 sum)

;; save the results of 1 step of decryption with the new

values

(mv-let (v0-final v1-final sum-final)

(tea-decrypt-step v0-new v1-new k0 k1 k2 k3 sum-new)

(and (equal v0-final v0)

(equal v1-final v1)

(equal sum-final sum))))))

Example Test Case:

(defthm test-case-1

(equal (mv-let (cipher-l cipher-r)

(tea-encrypt-wrapper #x12345678 #x9ABCDEF0

#xA56BABCD #x00000000 #xFFFFFFFF #x12345678)

(mv-let (plain-l plain-r)

(tea-decrypt-wrapper cipher-l cipher-r

#xA56BABCD #x00000000 #xFFFFFFFF #x12345678)

(list plain-l plain-r)))

(list #x12345678 #x9ABCDEF0)))

TODO Show the whole thing is invertible:

(defthm tea-encrypt-decrypt-invertible

(implies (All are u32...))

(mv-let (cipher-l cipher-r cipher-sum)

(tea-encrypt v0 v1 k0 k1 k2 k3 rounds sum)

(mv-let (plain-l plain-r plain-sum)

(tea-decrypt cipher-l cipher-r k0 k1 k2 k3

rounds cipher-sum)

(and (equal plain-sum sum)

(equal plain-l v0)

(equal plain-r v1)))

Potential Directions

- Shown a simple encryption algorithm where I know the encryption step is invertible can be inverted.
 - (enc x rounds) and (dec x rounds) is invertible
- Difficulty is that TEA has multiple parameters.
- Trying to “pack” the parameters together into a single x, show the x is invertible, and maybe I can apply the same idiom to prove the invertibility!